

GLA UNIVERSITY, GREATER NOIDA

Greater Noida – 201310, Uttar Pradesh, India

Community Health Risk Predictor: Disease Surge Forecasting System for Uttar Pradesh

SOFTWARE REQUIREMENT SPECIFICATION

Version 1.0

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING - AI

Submitted By

Vidushi Saraswat

Roll No.: 22515500100 | Section: NE

Under the Supervision of

Palash Raghuvanshi

GLA University, Greater Noida

SCHOOL / DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING / APPLICATIONS

GLA UNIVERSITY, GREATER NOIDA – 201310

September, 2026

Revision History

Version	Date	Author	Description
v1.0	04-Sep-2026	Vidushi Saraswat	Initial SRS draft

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Community Health Risk Predictor, a full-stack application that forecasts short-term dengue case trends for Uttar Pradesh and communicates the forecast as a calibrated probability rather than a single number. It is intended to guide development, and to give evaluators a falsifiable list of what the system will and will not do in this release.

1.2 Scope

The system ingests publicly available, state-level yearly dengue case and death data for Uttar Pradesh, fits a probability distribution to that data, trains a forecasting model, and serves both through a web dashboard. A user (student, clinic staff, or interested resident) can view historical case trends and a near-term risk estimate expressed as a probability with a confidence interval. The system does not perform clinical diagnosis, does not ingest real-time hospital data, and is limited to Uttar Pradesh in this release.

1.3 Definitions

Term	Definition
Surge	A period in which reported dengue cases exceed a defined threshold relative to the recent baseline.
Calibration	A simple check of whether a predicted probability turned out roughly right once the real number for that year is known.
Baseline	The historical case-count pattern used as the reference point against which a surge is measured.

2. Overall Description

2.1 User Classes

User Class	Description	Technical Level
General Resident	Views the dashboard to check current risk level for Uttar Pradesh.	Non-technical
Student/Developer	Uploads new data, retrains and evaluates the model.	Technical

2.2 Assumptions and Dependencies

- Publicly published state-level dengue data (data.gov.in, Kaggle) remains accessible for download.
- Uttar Pradesh case counts are reported annually, not weekly or daily, in the data currently available.
- The Python ML microservice and Java backend can communicate over local HTTP during development.

2.3 Constraints

C1 The system shall use only publicly available secondary data; it shall not connect to any hospital or clinical record system.

C2 The system shall be limited to Uttar Pradesh state-level data in this release.

3. Data Source and Baseline

3.1 Datasets Used

Field	Value
Dataset 1	data.gov.in – "State/UT-wise Dengue Cases and Deaths in the Country from 2018 to 2021" (Rajya Sabha Session 255, Unstarred Question No. 242)
Dataset 1 size	35 state/UT rows + 1 total row, 9 columns; no missing values (one blank serial number for Daman & Diu)
Dataset 1 UP figures	2018: 3,829 cases / 4 deaths — 2019: 10,557 / 26 — 2020: 3,715 / 6 — 2021: 27,506 / 8
Dataset 2	Kaggle – "Dengue Cases in India" (jadhavpranav), state-wise, 2019–2024 (2024 partial year)
Dataset 2 size	36 rows, 14 columns; the 2019_Cases column contains one non-numeric value ("NR" = Not Reported), which forces that column to load as text — identified as a required cleaning step
Dataset 2 UP figures	2019: 10,557 / 26 — 2020: 3,715 / 6 — 2021: 29,750 / 29 — 2022: 19,821 / 33 — 2023: 35,402 / 36 — 2024 (partial): 341 / 1
Granularity	State-level, yearly totals only — not district-level or monthly in either source
Known data quality issue	2021 UP case count differs between the two sources (27,506 vs. 29,750), most likely due to differing reporting cut-off dates; this will be documented rather than silently resolved

This baseline directly constrains the system: with only 4–6 yearly data points per source for Uttar Pradesh, the model in this release forecasts an annual trend and risk level, not a weekly or daily surge. This limitation is restated in Section 11 (Out of Scope).

4. System Architecture

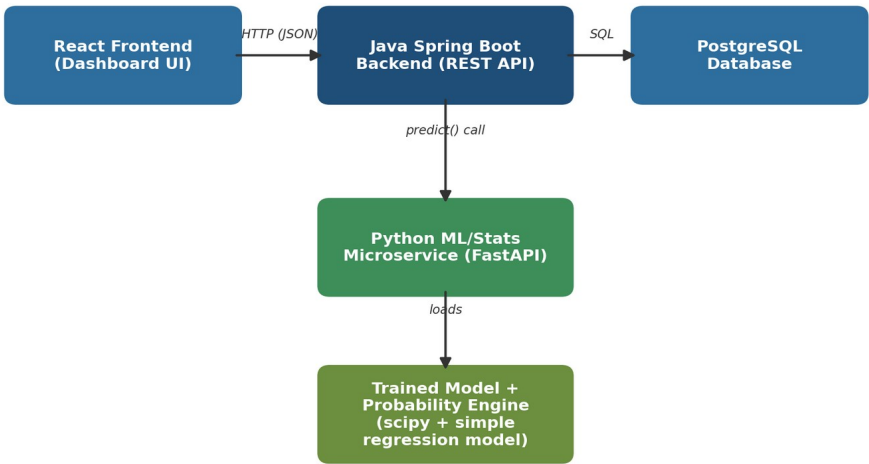


Figure 1: System architecture — React frontend, Java Spring Boot backend, Python ML/stats microservice, PostgreSQL database

Component	Responsibility
React Frontend	Displays historical trend chart, risk card, and a simple predicted-vs-actual comparison; calls the Java backend over HTTP
Java Spring Boot Backend	Exposes REST endpoints, manages PostgreSQL, calls the Python microservice for predictions
Python Microservice	Cleans data, fits probability distribution, trains/serves the ML forecasting model
PostgreSQL	Stores cleaned case data and a simple record of each prediction next to what actually

	happened, once that year's real number is known
--	---

5. Functional Requirements

5.1 Data Upload and Validation

FR-1.1 The system shall accept a CSV file of yearly, state-wise dengue case data via an upload endpoint.

FR-1.2 The system shall reject a CSV file missing the required columns (state name, year, case count), returning an error message naming the missing column.

FR-1.3 The system shall replace missing numeric case values using median imputation and record how many values were imputed.

FR-1.4 The system shall store the cleaned dataset in PostgreSQL, replacing the previous version for that data source.

5.2 Model-backed Requirements

FR-2.1 The system shall accept a request specifying a year and a case-count threshold, and reject a request with a threshold below zero with a 400 error.

FR-2.2 The system shall return a predicted case count and a 90% confidence interval for a valid request.

FR-2.3 The system shall present the probability that cases exceed the requested threshold together with the confidence interval width.

FR-2.4 (stretch, optional) The system shall, when the Python microservice is unavailable, return the most recent stored prediction with a timestamp instead of failing silently.

FR-2.5 (stretch, optional) The system shall allow a Student/Developer user to trigger a model retrain using the currently stored dataset.

6. Non-Functional Requirements

NFR-1 The system shall return a prediction result within 3 seconds of a valid /predict request under normal load.

NFR-2 The system shall require a fixed key value in the request header for the /upload and /retrain endpoints, rejecting requests without it.

NFR-3 The system shall log every prediction request with a timestamp and a unique request ID in PostgreSQL.

NFR-4 The system shall remain usable by a non-technical user without documentation, verified by having one person outside the project complete a risk lookup unassisted.

7. Build Plan

7.1 Minimum Viable Product

The MVP is: FR-1.1, FR-1.2, FR-1.3, FR-2.1, FR-2.2, FR-2.3.

In simple terms: the MVP is the smallest version of the project that still works start to finish — upload some data, clean it, and get back a real prediction with a probability attached. These six requirements are chosen because they form one complete, working path through the system. Nothing else (the dashboard, retraining, extra polish) is useful until this path exists, so it is built first.

FR-2.4 and FR-2.5 are marked as optional/stretch and left out of the MVP on purpose — they add convenience (fallback behaviour, retraining on demand) once the core path already works, but are not needed for the project to be considered complete and working.

7.2 Build Order

Target completion: December 2026. Stages below are spaced to leave a buffer before the final deadline — adjust the exact dates if your course gives fixed checkpoint dates.

Stage	Target Date	Requirements Targeted
Data collection & cleaning	20-Sep-2026	FR-1.1, FR-1.2, FR-1.3, FR-1.4
Statistics & probability layer	10-Oct-2026	FR-2.1, FR-2.2, FR-2.3
ML model & backend integration	10-Nov-2026	FR-2.2, FR-2.4
Frontend & dashboard	25-Nov-2026	FR-2.2, FR-2.3 (display)
Retrain workflow & polish	10-Dec-2026	FR-2.5, NFR-1–NFR-4

8. Data Requirements

8.1 Entities

Entity: DengueCaseRecord

Field	Type	Notes
state	text	Fixed as "Uttar Pradesh" in this release
year	integer	2018–2024 depending on source
cases	integer	Median-imputed if originally missing/non-numeric
deaths	integer	As reported
source	text	"data.gov.in" or "Kaggle", to trace discrepancies

Other entities: PredictionLog (stores each prediction and, once known, the real number for that year, so the two can be compared side by side); User (Student/Developer login for upload and retrain access).

8.2 Entity Relationship Diagram

NOT REQUIRED FOR v1.0 — to be completed in a later version.

9. Design Decisions and Alternatives Considered

Each choice below is written so it can be explained out loud in one or two plain sentences — that is the version to practice saying, not the table itself.

Decision	Alternative Considered	Reason for Choice (simple)
Split the backend into two parts: Java handles the app and database; Python handles the maths/ML	Write the whole backend in just one language (only Python)	The project needs to show both Java and Python. Splitting the work this way lets each language do the part it is naturally better at, instead of forcing everything into one file.
Use a counting-based formula (Poisson) to estimate disease case probabilities	Use a plain average / bell-curve style estimate	Case counts are always whole numbers, never negative, and can spike a lot in bad years — a plain average doesn't handle that well. Poisson is a standard formula made exactly for counting things like this. (See the plain-language explanation below the table.)
Start with yearly state-level	Delay the whole project until more	Only yearly numbers were available publicly in

data instead of waiting for more detailed data	detailed (weekly/district) data is found	the time we had. Waiting was not realistic before the deadline, and yearly data is still enough to show a clear rising/falling trend, which is the goal for this version.
--	--	---

Simple explanation for the Poisson choice: disease case counts are always whole numbers and can never go below zero, and some years spike much higher than others. A plain average treats big spikes as "unusual errors," but Poisson is a formula built specifically for counting events like this, so it gives a more honest probability instead of an unrealistic one. If asked to explain this live, it is fine to say exactly this in your own words — you do not need the formula memorised, just the idea that it is "a counting formula built for things like disease cases, instead of a generic average."

10. Acceptance Criteria

AC-1 Given a valid CSV upload, when submitted via the /upload endpoint, the system stores the cleaned data and returns a success response listing the number of rows imputed.

AC-2 Given a valid /predict request for a specified year and threshold, the system returns a case-count prediction, confidence interval, and probability within 3 seconds.

AC-3 Given the Python microservice is stopped, when a /predict request is made, the system returns the last stored prediction with its timestamp instead of a server error.

AC-4 Given a CSV upload missing the "cases" column, when submitted, the system rejects it and returns an error naming that column.

11. Out of Scope for This Release

Real-time hospital/clinical data integration — not included. Reason: this project only uses data that is already public (government/Kaggle files); it does not connect to any live hospital system, which a student project would not have access to anyway.

District-level or weekly/monthly forecasting — not included. Reason: the data we found only gives yearly totals for the whole state, not smaller areas or shorter time periods, so the system can only forecast at that same yearly level.

Covering more than one state or more than one disease — not included. Reason: keeping the project to one state (Uttar Pradesh) and one disease (dengue) keeps it achievable in the given time. The way the system is built does not block adding more states or diseases later — it is simply not attempted in this version.

Appendix A: State Model

NOT REQUIRED FOR v1.0 — to be completed in a later version. Delete this appendix if nothing in the system has a lifecycle.

Appendix B: Traceability

NOT REQUIRED FOR v1.0 — to be completed once task-board and test-case IDs exist.